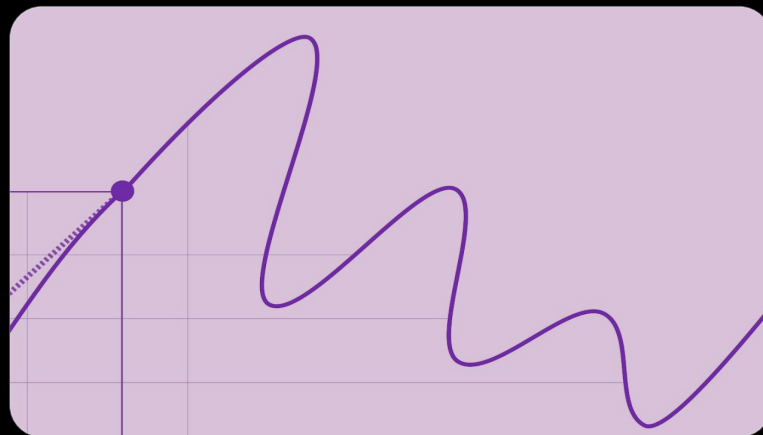
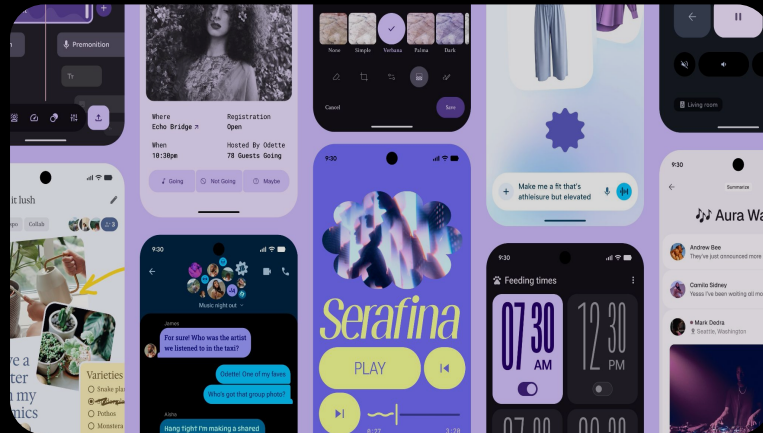


Dynamic List

Prof. Dr. Fernando Sambinelli
IFSP Hortolândia

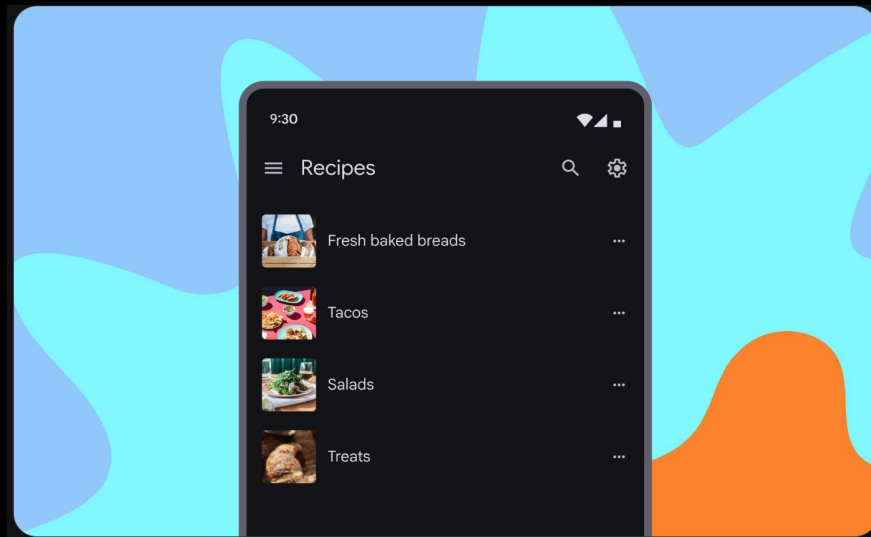
Vamos Começar



Dynamic List

São composables que exibem coleções de dados que podem mudar ao longo do tempo, permitindo que itens sejam adicionados, removidos ou alterados automaticamente, mantendo a UI sincronizada com os dados subjacentes

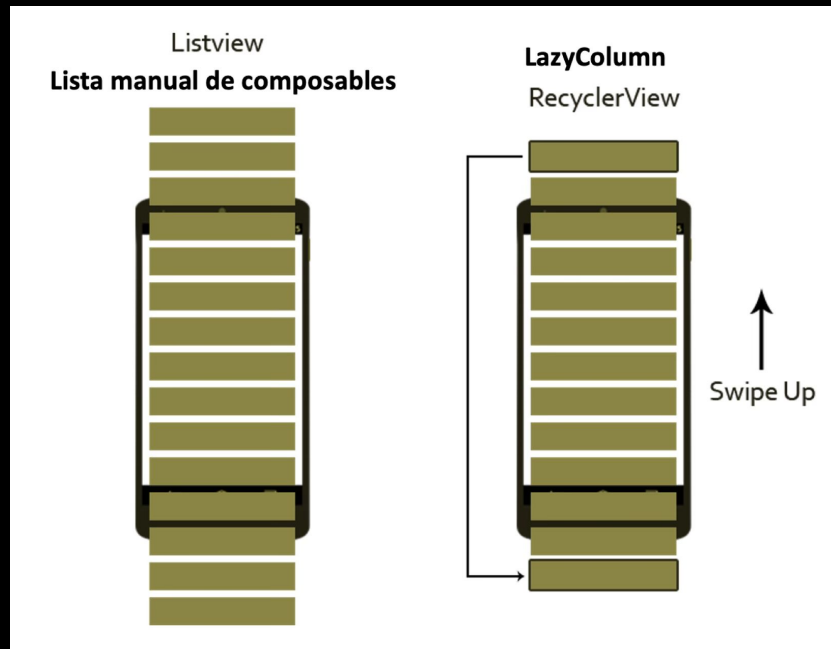
- * Otimizadas para listagem de grandes volumes de dados
- * Carregamento mais eficiente



Modos de Carregamentos

Manual e LazyColumn

- * **Manual:** Cria-se manualmente cada item da lista, um por um, o que pode ser ineficiente para listas grandes, pois todos os itens são renderizados de uma vez.
- * **LazyColumn:** apenas os itens visíveis na tela são compostos e renderizados automaticamente conforme o usuário rola a lista, tornando o carregamento mais eficiente e econômico em recursos.



Carregamento Manual (1)

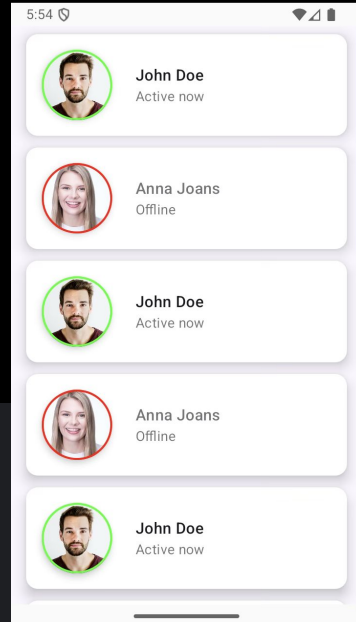
```
UserProfile.kt
1 package com.sambinelli.carregamentomanualdelistadinamica
2
3 data class UserProfile (val name: String, val status: Boolean, val drawableId: Int)
4
5 val userProfileList = arrayListOf(
6     UserProfile(name = "John Doe", status = true, R.drawable.profile_picture),
7     UserProfile(name = "Anna Jones", status = false, R.drawable.profile_picture2),
8     UserProfile(name = "John Doe", status = true, R.drawable.profile_picture),
9     UserProfile(name = "Anna Jones", status = false, R.drawable.profile_picture2),
10    UserProfile(name = "John Doe", status = true, R.drawable.profile_picture),
11    UserProfile(name = "Anna Jones", status = false, R.drawable.profile_picture2),
12    UserProfile(name = "John Doe", status = true, R.drawable.profile_picture),
13    UserProfile(name = "Anna Jones", status = false, R.drawable.profile_picture2),
14    UserProfile(name = "John Doe", status = true, R.drawable.profile_picture),
15    UserProfile(name = "Anna Jones", status = false, R.drawable.profile_picture2),
16    UserProfile(name = "John Doe", status = true, R.drawable.profile_picture),
17    UserProfile(name = "Anna Jones", status = false, R.drawable.profile_picture2),
18 )
```

```
46 class MainActivity : AppCompatActivity() {
47     override fun onCreate(savedInstanceState: Bundle?) {
48         super.onCreate(savedInstanceState)
49         enableEdgeToEdge()
50         setContentView {
51             CarregamentoManualDeListaDinamicaTheme {
52                 MainScreen()
53             }
54         }
55     }
56 }
57
58 @Composable
59 fun MainScreen() {
60     Surface(
61         modifier = Modifier
62             .fillMaxSize()
63             .safeDrawingPadding(),
64     ) {
65         val scrollState = rememberScrollState()
66         Column(
67             modifier = Modifier.verticalScroll(scrollState)
68         ) {
69             for (userProfile in userProfileList) {
70                 ProfileCard(userProfile = userProfile)
71             }
72         }
73     }
74 }
```

Carregamento Manual (2)

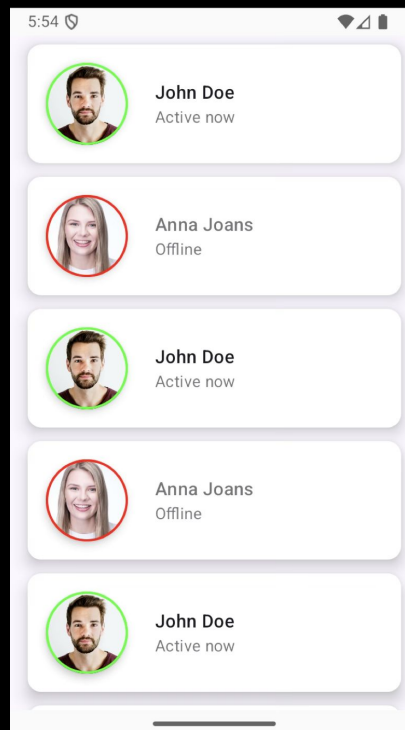
```
71 @Composable
72 fun ProfileCard(userProfile: UserProfile) {
73     Card(
74         modifier = Modifier
75             .padding(top = 8.dp, bottom = 4.dp, start = 16.dp, end = 16.dp)
76             .fillMaxWidth()
77             .wrapContentHeight(),
78         elevation = CardDefaults.cardElevation(defaultElevation = 8.dp),
79         colors = CardDefaults.cardColors(containerColor = Color.White)
80     ) {
81         Row(
82             modifier = Modifier.fillMaxWidth(),
83             verticalAlignment = Alignment.CenterVertically,
84             horizontalArrangement = Arrangement.Start
85         ) {
86             ProfilePicture(userProfile.drawableId, userProfile.status)
87             ProfileContent(userProfile.name, userProfile.status)
88         }
89     }
90 }
91
92 @Composable
93 fun ProfilePicture(drawableId: Int, onlineStatus: Boolean) {
94     Card(
95         shape = CircleShape,
96         border = BorderStroke(
97             width = 2.dp,
98             color = if (onlineStatus)
99                 Color.Green
100             else Color.Red
101         ),
102         modifier = Modifier.padding(16.dp),
103         elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)
104     ) {
105         Image(
106             painter = painterResource(id = drawableId),
107             modifier = Modifier.size(72.dp),
108             contentScale = ContentScale.Crop,
109             contentDescription = "Descrição da imagem."
110         )
111     }
112 }
```

```
114 @Composable
115 fun ProfileContent(userName: String, onlineStatus: Boolean) {
116     Column(
117         modifier = Modifier
118             .padding(8.dp)
119             .fillMaxWidth()
120     ) {
121         Text(
122             text = userName,
123             style = MaterialTheme.typography.titleMedium,
124             color = if (onlineStatus)
125                 MaterialTheme.colorScheme.onSurface
126             else MaterialTheme.colorScheme.onSurface.copy(alpha = 0.6f)
127         )
128         Text(
129             text = if (onlineStatus) "Active now" else "Offline",
130             style = MaterialTheme.typography.bodyMedium,
131             color = MaterialTheme.colorScheme.onSurface.copy(alpha = 0.6f)
132         )
133     }
134 }
135
136 @Preview(showBackground = true)
137 @Composable
138 fun GreetingPreview() {
139     CarregamentoManualDeListaDinamicaTheme {
140         MainScreen()
141     }
142 }
```



Carregamento LazyColumn

```
58 @Composable
59 fun MainScreen() {
60     Surface(
61         modifier = Modifier
62             .fillMaxSize()
63             .safeDrawingPadding(),
64     ) {
65         // val scrollState = rememberScrollState()
66         // Column(
67             //     modifier = Modifier.verticalScroll(scrollState)
68         // ) {
69             // for (userProfile in userProfileList) {
70                 //     ProfileCard(userProfile = userProfile)
71             // }
72         // }
73
74         LazyColumn() {
75             items(userProfileList) { userProfile ->
76                 ProfileCard(userProfile = userProfile)
77             }
78         }
79     }
80 }
81 }
```



Carregamento de Imagem

Coil Library



- * **Coil Library:** A Coil Library é uma biblioteca de carregamento de imagens para Android que simplifica e otimiza o processo de exibir imagens em apps, fornecendo uma forma fácil e eficiente de carregar, cachear e exibir imagens de URLs, recursos locais e mais
 - * https://coil-kt.github.io/coil/getting_started/
 - * Adicionar dependências:
 - `implementation("io.coil-kt.coil3:coil-compose:3.4.0")`
 - `implementation("io.coil-kt.coil3:coil-network-okhttp:3.4.0")`

libs.version.toml e gradle

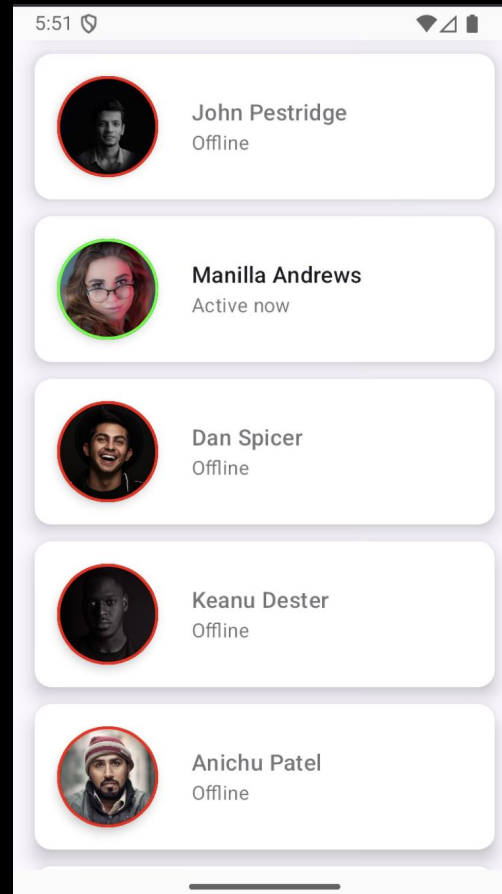
```
1 [versions]
2 agp = "8.8.2"
3 kotlin = "2.2.0"
4 coreKtx = "1.16.0"
5 junit = "4.13.2"
6 junitVersion = "1.3.0"
7 espressoCore = "3.7.0"
8 lifecycleRuntimeKtx = "2.9.3"
9 activityCompose = "1.10.1"
10 composeBom = "2024.04.01"
11 coil = "3.4.0"
12
13 [libraries]
14 androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
15 junit = { group = "junit", name = "junit", version.ref = "junit" }
16 androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
17 androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
18 androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref = "lifecycleRuntimeKtx" }
19 androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }
20 androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }
21 androidx-ui = { group = "androidx.compose.ui", name = "ui" }
22 androidx-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
23 androidx-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
24 androidx-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
25 androidx-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
26 androidx-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
27 androidx-material3 = { group = "androidx.compose.material3", name = "material3" }
28
29 coil-compose = { group = "io.coil-kt.coil3", name = "coil-compose", version.ref = "coil" }
30 coil-network-okhttp = { group = "io.coil-kt.coil3", name = "coil-network-okhttp", version.ref = "coil" }
31
32 [plugins]
33 android-application = { id = "com.android.application", version.ref = "agp" }
```

```
40 }
41
42 dependencies {
43
44     implementation(libs.androidx.core.ktx)
45     implementation(libs.androidx.lifecycle.runtime.ktx)
46     implementation(libs.androidx.activity.compose)
47     implementation(platform(dependencyProvider = libs.androidx.compose.bom))
48     implementation(libs.androidx.ui)
49     implementation(libs.androidx.ui.graphics)
50     implementation(libs.androidx.ui.tooling.preview)
51     implementation(libs.androidx.material3)
52
53     implementation(libs.coil.compose)
54     implementation(libs.coil.network.okhttp)
55
56     testImplementation(libs.junit)
57     androidTestImplementation(libs.androidx.junit)
58     androidTestImplementation(libs.androidx.espresso.core)
59     androidTestImplementation(platform(dependencyProvider = libs.androidx.compose.bom))
60     androidTestImplementation(libs.androidx.ui.test.junit4)
61     debugImplementation(libs.androidx.ui.tooling)
62     debugImplementation(libs.androidx.ui.test.manifest)
63 }
```

LazyColumn via URL

Adicionar Permissão

```
AndroidManifest.xml x
1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3    xmlns:tools="http://schemas.android.com/tools">
4
5    <uses-permission android:name="android.permission.INTERNET"/>
6
7    <application
8        android:allowBackup="true"
9        android:dataExtractionRules="@xml/data_extraction_rules"
10        android:fullBackupContent="@xml/backup_rules"
11        android:icon="@mipmap/ic_launcher"
12        android:label="Carregamento Manual de Lista Dinamica"
13        android:roundIcon="@mipmap/ic_launcher_round"
14        android:supportsRtl="true"
15        android:theme="@style/Theme.CarregamentoManualDeListaDinamica"
16        tools:targetApi="31">
17        <activity
18            android:name=".MainActivity"
19            android:exported="true"
20            android:label="Carregamento Manual de Lista Dinamica"
21            android:theme="@style/Theme.CarregamentoManualDeListaDinamica">
22            <intent-filter>
23                <action android:name="android.intent.action.MAIN" />
24
25                <category android:name="android.intent.category.LAUNCHER" />
26            </intent-filter>
27        </activity>
28    </application>
```





Fernando Sambinelli

Professor e Doutor em Tecnologia
sambinelli@ifsp.edu.br



INSTITUTO FEDERAL
São Paulo

